

System Design Basics

Lesson 3: Understanding Load and Traffic

Understanding Load and Traffic – Study Notes

Key Concepts

1. **QPS (Queries /Per /Second)** – the number of requests a system receives each second.
2. **Latency** – the time taken to process a single request (often measured in ms).
3. **Throughput** – the amount of work completed per unit time (e.g., messages delivered per second).
4. **Load** – the combined effect of QPS, latency, and resource usage on a system.
5. **Capacity Planning** – estimating the resources required to handle expected load with acceptable performance.

Summary

Load and traffic are fundamental metrics for designing, operating, and scaling any online service.

QPS tells you how many requests hit your service, while **latency** tells you how long each request takes to complete. **Throughput** is the product of these two (or, more precisely, the number of successful operations per second) and reflects the real work the system is doing.

When building a chat application, you can calculate the expected load by multiplying the number of active users, the average messages each user sends per minute, and the conversion to seconds. This gives you the required QPS. Knowing the target latency (e.g., < /100 /ms for a responsive UI) lets you estimate the necessary processing capacity and network bandwidth. These calculations guide decisions about server sizing, load balancer configuration, and scaling strategies.

Important Points

- **QPS = Total Requests / Time Window**
- **Latency** can be broken into *network latency*, *processing latency*, and *queueing latency*.
- **Throughput** "H $\text{QPS} \times \text{Success Rate}$ " (or messages per second for a chat app).
- **Load = QPS $\times$ Average Latency** ' gives an idea of concurrent work in the system (e.g., number of threads or connections needed).
- **Peak vs. average load** – always design for peak traffic plus a safety margin (commonly 20 30 %).
- **Back pressure** mechanisms (circuit breakers, rate limiting) protect the system when load spikes.
- **Monitoring**: track QPS, latency (p50/p95/p99), error rates, and resource utilization (CPU, memory, network).

Examples

Example 1 – Calculating QPS for a Chat App

Assume:

- 10 /000 registered users.
- 5 /% are active at any given minute !' 500 active users.
- Each active user sends an average of 2 messages per minute.

Step 1 – Total messages per minute

$(500 \text{ users} \times 2 \text{ msgs/min}) = 1000 \text{ msgs/min}$

Step 2 – Convert to QPS

$(1000 \text{ msgs/min}) \div 60 \text{ sec/min} = 16.7 \text{ QPS}$

So the chat service must handle roughly **17 QPS** under normal conditions.

If you want to support a 2× traffic spike, provision for **34 QPS** plus a safety margin.

Example 2 – Relating Latency to Required Capacity

Target latency for message delivery: **100 ms**.

Assume each server can process **1 /000 requests per second** with that latency.

- Required QPS (from Example 1, peak) = 34 QPS.
- One server can handle 1 /000 QPS, so a single instance is sufficient for processing.

However, you also need to consider:

- **Network I/O** (messages per second × average message size).
- **Redundancy** (at least 2 instances for high availability).

Thus, you would typically deploy **2 small instances** behind a load balancer, giving headroom and fault tolerance.

Quick Review

1. **What does QPS stand for and how is it calculated?**
2. **If a system processes 200 requests per second with an average latency of 50 /ms, what is its approximate throughput?**
3. **How would you compute the expected QPS for a chat app with 8 /000 users, 10 /% active, each sending 3 messages per minute?**
4. **Why is it important to design for peak load rather than just average load?**
5. **Name two monitoring metrics you would watch to detect load related problems.**

Further Reading

- **Capacity Planning & Autoscaling** – how to use cloud auto scaling groups based on QPS/CPU thresholds.
- **Latency Breakdown** – deep dive into network vs. processing latency and tools to measure them (e.g., tracing, ping, curl).
- **Load Testing Tools** – Intro to Locust, k6, JMeter for generating synthetic traffic and measuring QPS/latency.
- **Back Pressure Patterns** – circuit breakers, token buckets, and rate limiting in high traffic services.
- **Distributed Tracing** – using OpenTelemetry or Zipkin to follow a request through a microservice architecture.

End of notes.

